



Administrator Linux. Professional

Динамический веб



Проверить, идет ли запись

Меня хорошо видно && слышно?



Преподаватель



Лавлинский Николай

Технический директор «Метод Лаб»

Более 15 лет в веб-разработке

Преподавал в ВУЗе более 10 лет
Более 5 лет в онлайн-образовании

Специализация: оптимизация производительности, ускорение сайтов и веб-приложений

https://t.me/methodlab_tg

<https://www.methodlab.ru/>

<https://rutube.ru/channel/24617406/>

<https://www.youtube.com/c/NickLavlinsky>

https://www.youtube.com/@site_support

<https://vkvideo.ru/@methodlab>

<https://habr.com/ru/users/Nickmob/>

Правила вебинара



Активно
участвуем



Off-topic обсуждаем
в Телеграм-чате



Задаем вопрос
в чат или голосом



Вопросы вижу в чате,
могу ответить не сразу

Условные обозначения



Индивидуально



Время, необходимое
на активность



Пишем в чат



Говорим голосом

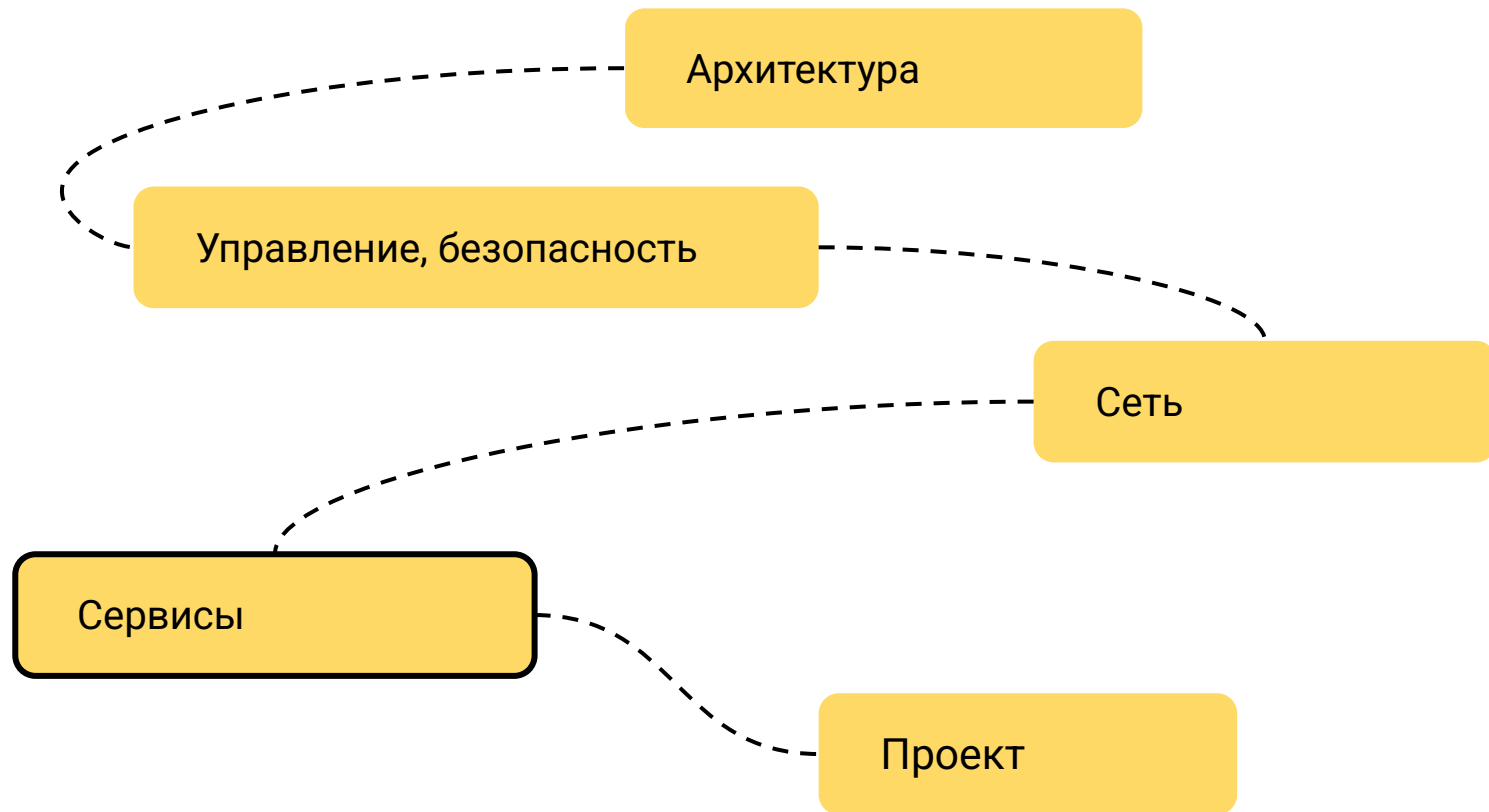


Документ

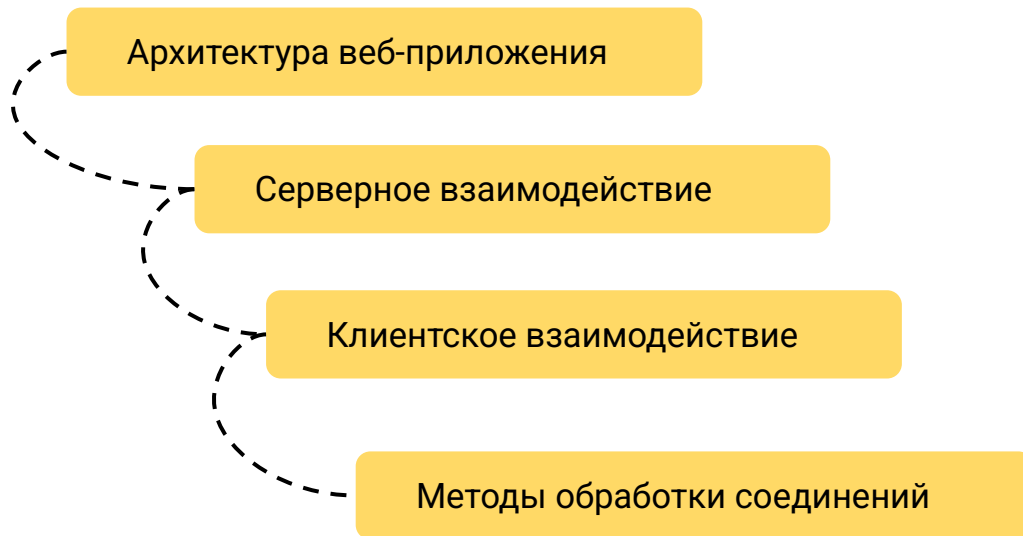


Ответьте себе или
задайте вопрос

Карта курса



Маршрут вебинара



Цели вебинара

После занятия вы сможете

1. Понимать варианты взаимодействия компонентов веб-приложения
2. Выбирать эффективную архитектуру
3. Различать методы обработки соединений

Смысл

Зачем вам это уметь

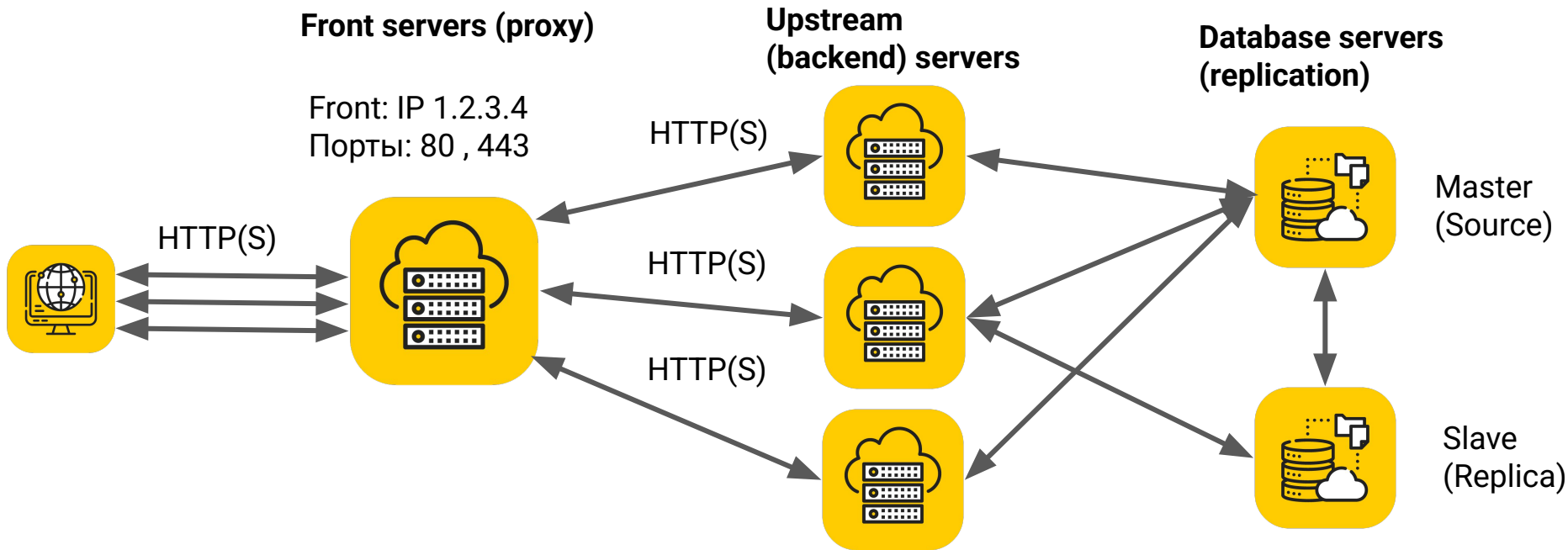
1. Запускать веб-приложения
2. Решать проблемы и отлаживать системы
3. Оптимизировать производительность



Архитектура веб-приложений



Стандартная архитектура приложения



Протоколы взаимодействия

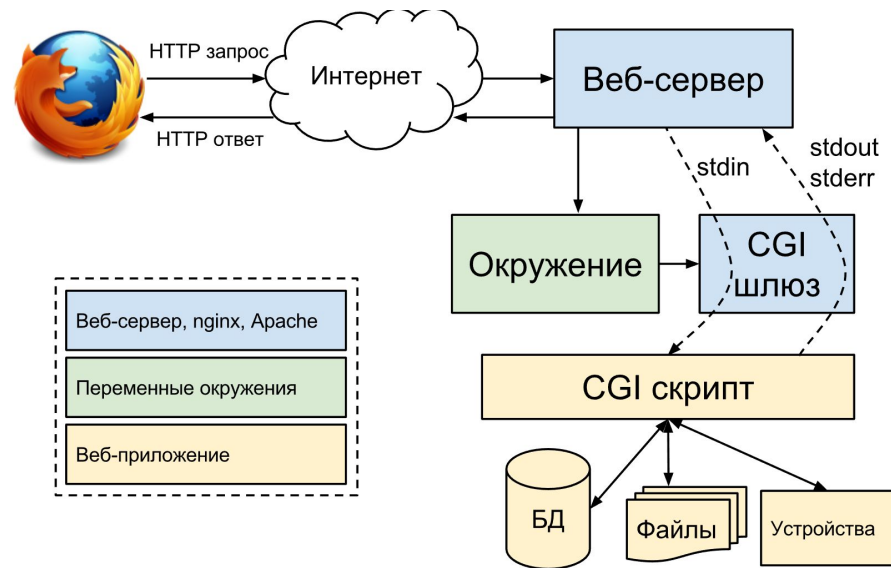
- Протоколы взаимодействия с сервером приложений
 - CGI
 - FastCGI
 - SCGI
 - WSGI
- Клиентские протоколы (браузер-сервер)
 - AJAX (XHR)
 - WebSocket

Протоколы взаимодействия с сервером приложений



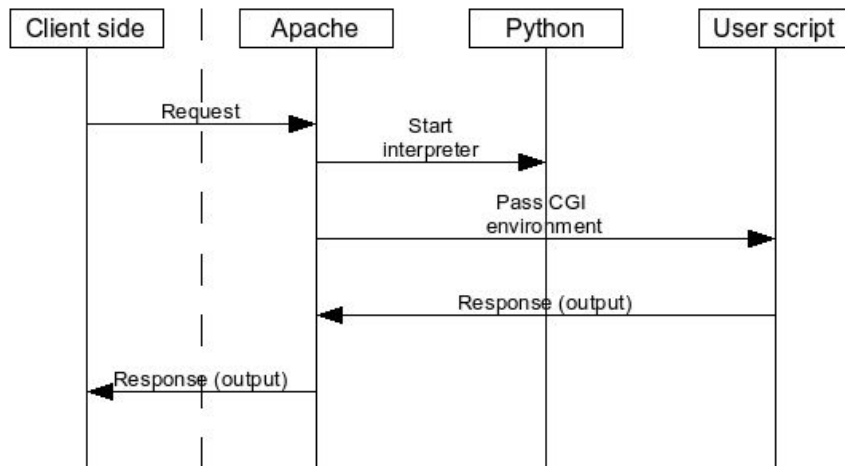
CGI

- CGI (Common Gateway Interface) - стандарт интерфейса, используемого для связи внешней программы с веб-сервером
- Скрипт запускается при каждом запросе
- CGI-параметры передаются в виде переменных окружения
- Скрипт возвращает результат в STDOUT, который транслируется сервером клиенту
- Высокие накладные расходы на интерпретацию кода (каждый запрос)



CGI – процесс обработки запроса

CGI



CGI — конфигурация (Apache)

```
LoadModule cgi_module /usr/lib/apache2/modules/mod_cgi.so
```

```
AddHandler cgi-script .cgi .pl
```

```
ScriptAlias /cgi-bin/ /home/www/cgi-bin/
```

<https://httpd.apache.org/docs/2.4/howto/cgi.html>



FastCGI

- Протокол взаимодействия между веб-сервером и сервером приложений
- Блокирующий доступ
- Используется TCP или UNIX socket
- Пример конфигурации в Nginx:

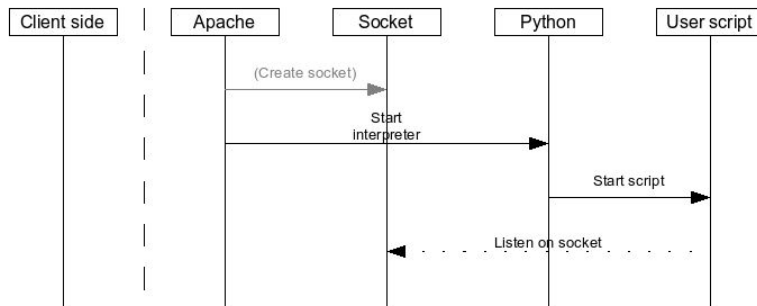
```
location ~ ^/(status|ping)$ {  
    allow 127.0.0.1;  
    deny all;  
    fastcgi_param SCRIPT_FILENAME $document_root$fastcgi_script_name;  
    fastcgi_index index.php;  
    include fastcgi_params;  
    fastcgi_pass unix:/var/run/phpfpm-api.sock;  
}
```

FastCGI — принципы работы

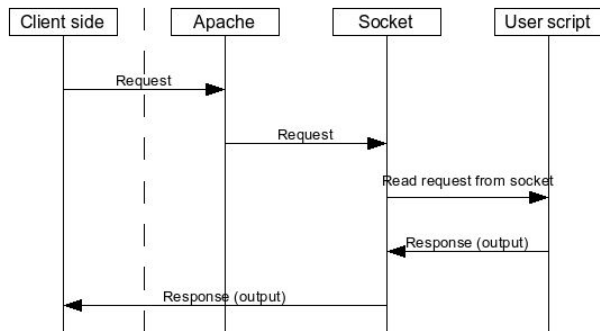
- Программа загружается в память в качестве при первом обращении к серверу
- Не требуется процесс интерпретации и подготовки кода к исполнению
- Один и тот же процесс обрабатывает множество запросов
- Высокая производительность
- Больше потребление оперативной памяти

FastCGI — процесс обработки запроса

FastCGI (Startup)

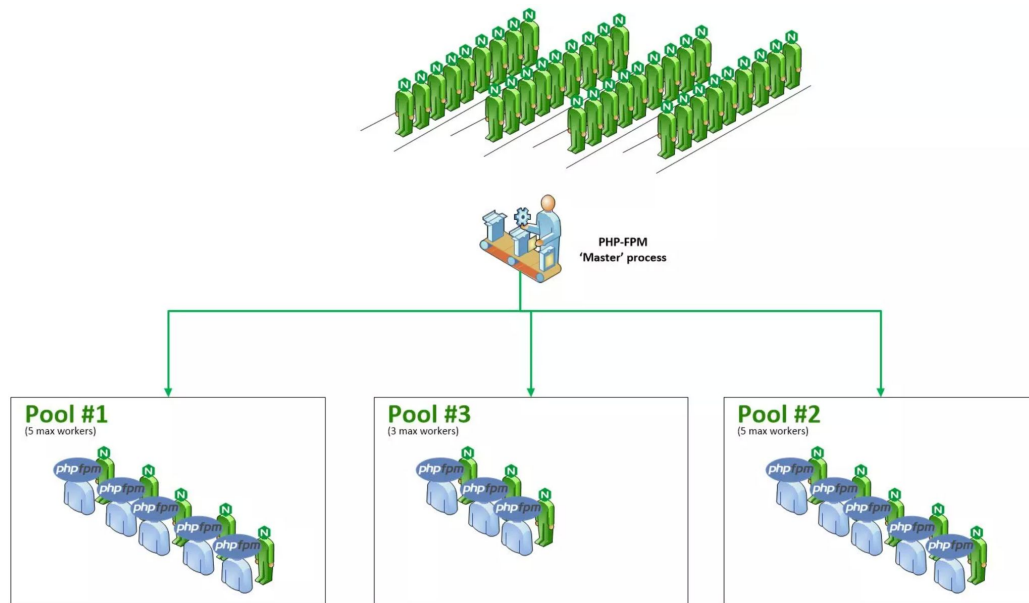


FastCGI (Request handling)



PHP-FPM

- Популярная реализация FastCGI-сервера для PHP
- Работает с большинством веб-приложений
- Позволяет настраивать пулы для приложений
- Статическое и динамическое количество воркеров

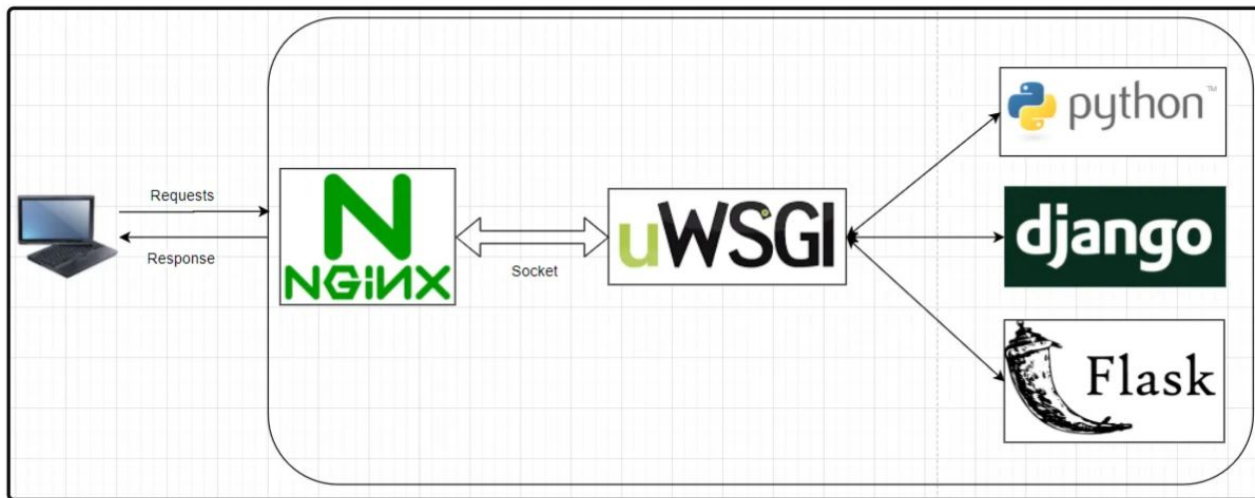


WSGI framework

- Стандарт взаимодействия между Python-программой, выполняющейся на стороне сервера, и самим веб-сервером
- Должно быть вызываемым (callable) объектом (обычно это функция или метод)
- Параметры на входе
 - Словарь переменных окружения (environ)
 - Обработчик запроса (start_response)
- Вызывает обработчик запроса с кодом HTTP-ответа и HTTP-заголовками
- Возвращает итерируемый объект с телом ответа

uWSGI

- uWSGI — веб-сервер и сервер веб-приложений, первоначально реализованный для запуска приложений Python через протокол WSGI (и его бинарный вариант uwsgi)
- Версия 2.0 поддерживает также запуск веб-приложений Lua, Perl, Ruby и других



uWSGI

```
location / {  
    include      uwsgi_params;  
    uwsgi_pass   localhost:81;  
}
```

http://nginx.org/en/docs/http/nginx_http_uwsgi_module.html

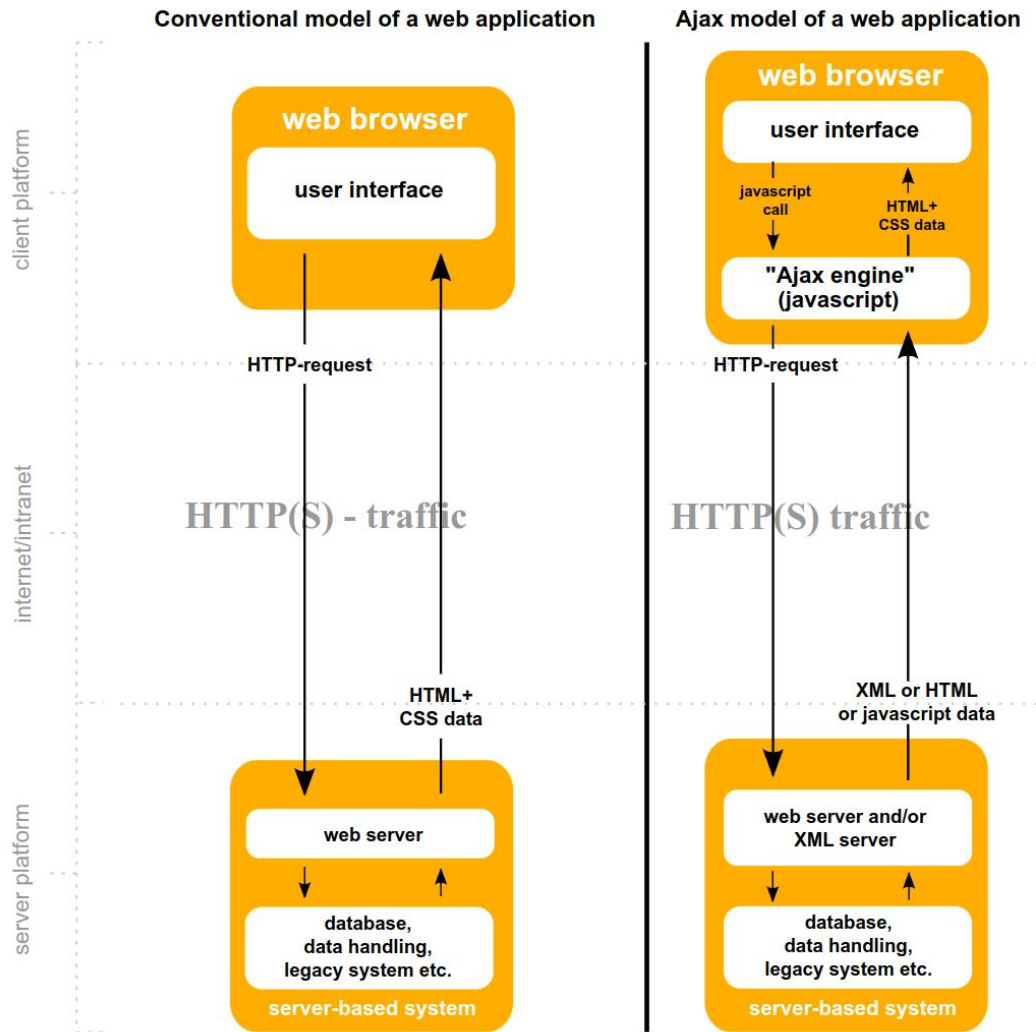
Клиентские протоколы (браузер-сервер)



AJAX (XHR)

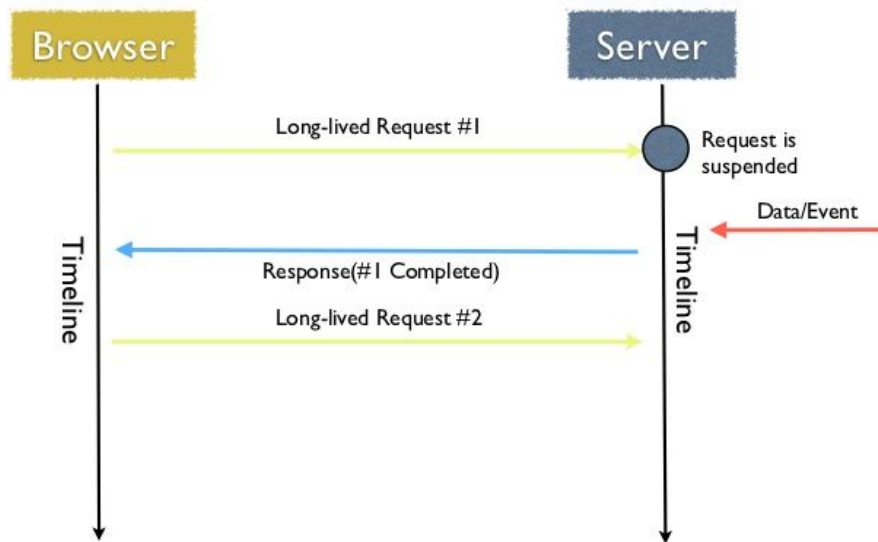
- Asynchronous Javascript and XML — «асинхронный JavaScript и XML»
- XMLHttpRequest (XMLHTTP, XHR) — API в JS для отправки HTTP-запросов
- fetch — современная версия API в JS
- Отправка запросов без перезагрузки страницы
- Обновление частей страницы
- Динамическая загрузка (бесконечная прокрутка)
- Данные в формате JSON или HTML

AJAX



AJAX Long Polling

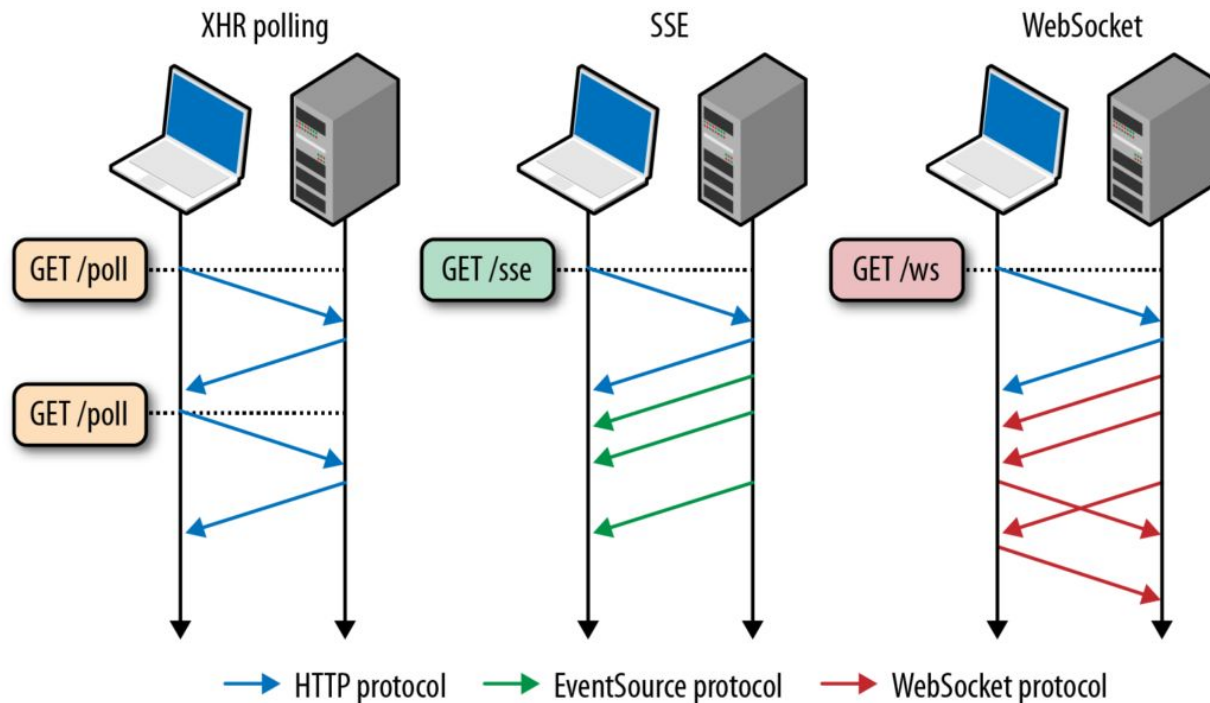
Comet:Http Long Polling



WebSocket

- Протокол для долгоживущих соединений поверх TCP
- Соединение устанавливается с помощью HTTP
- Сообщения могут идти в обе стороны
- Схема URL:
 - HTTP: ws://ws.my.site
 - HTTPS: wss://wss.my.site

WebSocket vs XHR vs SSE (Server-sent events)



WebSocket в Nginx

```
http {  
    map $http_upgrade $connection_upgrade {  
        default upgrade;  
        '' close;  
    }  
  
    ...  
  
    location /wsapp/ {  
        proxy_pass http://wsbackend;  
        proxy_http_version 1.1;  
        proxy_set_header Upgrade $http_upgrade;  
        proxy_set_header Connection $connection_upgrade;  
        proxy_set_header Host $host;  
    }  
}
```

<https://www.f5.com/company/blog/nginx/websocket-nginx>



Методы обработки соединений



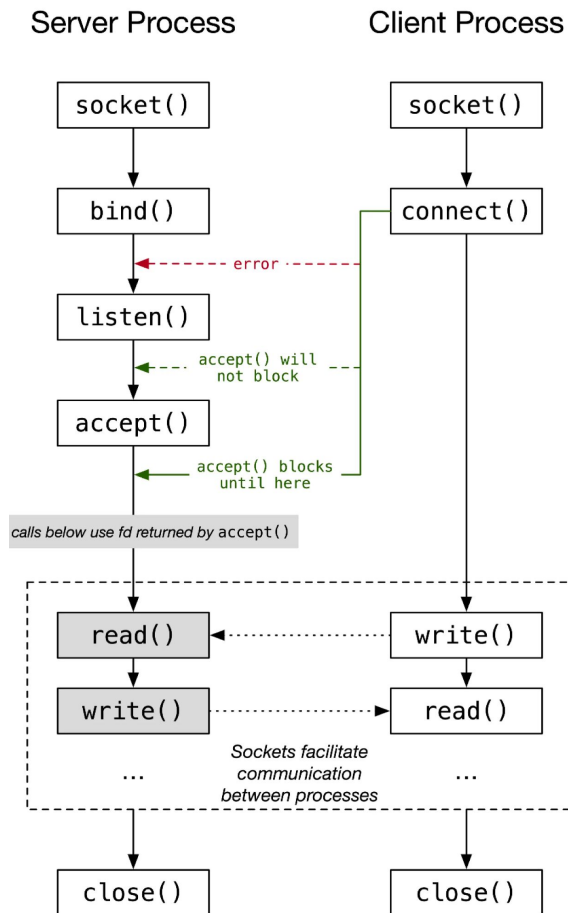
Обработка соединений

- Один клиент
 - UNIX-сокеты
 - TCP-сокеты
- Мультиплексирование
 - `select()`
 - `epoll()`

<http://nginx.org/ru/docs/events.html>

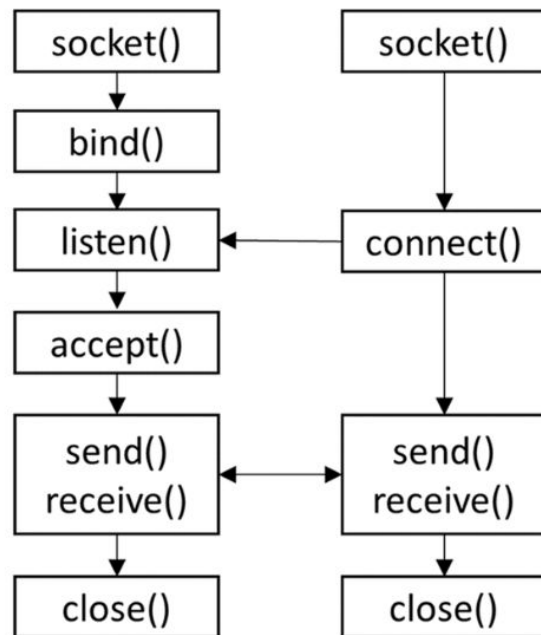


Unix-socket

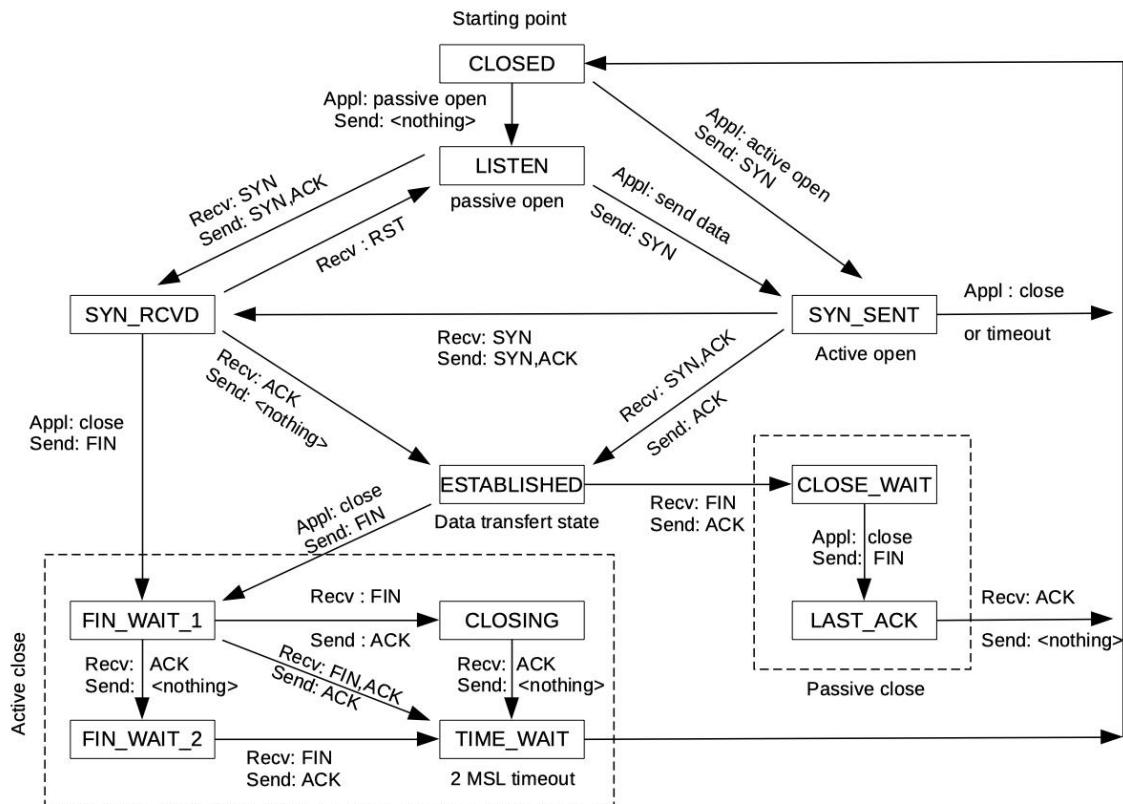


TCP-socket

TCP Server TCP Client



Состояния ТСР-сокета



Обработка соединений select()

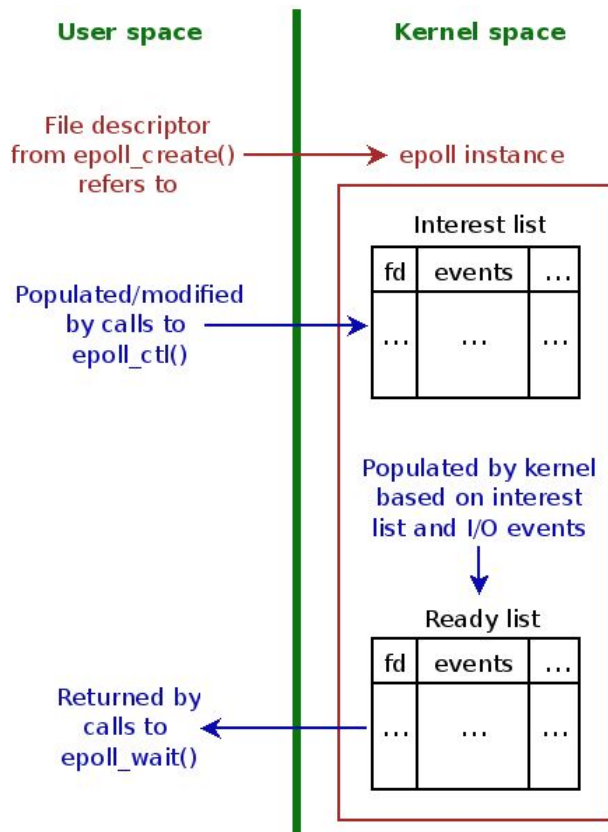
- На каждом вызове опрашиваются все файловые дескрипторы на предмет изменения статуса
- Ограниченный набор дескрипторов (select())
- Псевдокод обработки с select()

```
select(socket);  
while (1) {  
    sockets = select();  
    for (socket in sockets) {  
        if (can_read(socket)) {  
            read(socket, buffer);  
            process(buffer);  
        }  
    }  
}
```

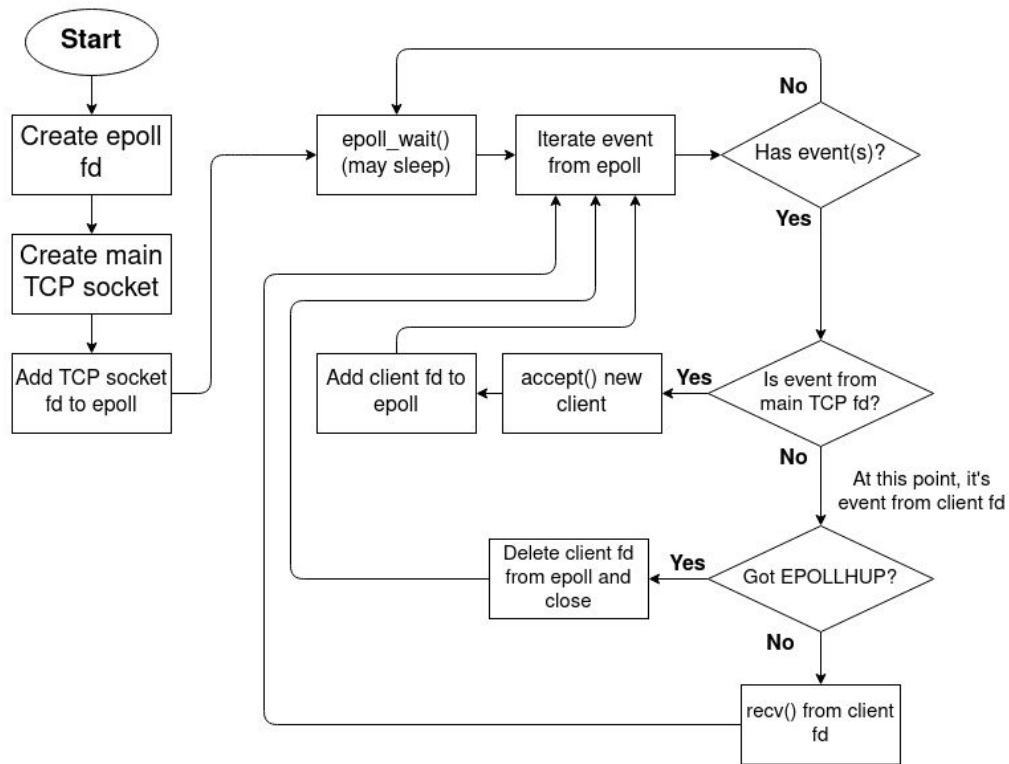
Обработка соединений `epoll()`

- API мультиплексированного ввода-вывода
- Позволяет приложениям осуществлять мониторинг нескольких открытых файловых дескрипторов
- Более эффективная замена вызовам `select()` и `poll()`, определёнными в POSIX
- Может предоставить более эффективный механизм для приложений, обрабатывающих большое количество одновременно открытых соединений — со сложностью $O(1)$
- `/proc/sys/fs/epoll/max_user_watches` - максимальное количество наблюдаемых fd для каждого RealUID

epoll()



epoll()



Вопросы?



Ставим “+”,
если вопросы есть



Ставим “-”,
если вопросов нет

Практика

Тест

Вопросы для проверки

Какой API мультиплексированного ввода-вывода планировался как более эффективная замена вызовам `select()` и `poll()` и позволяет обрабатывать большое количество одновременных соединений со сложностью $O(1)$?

- A. `epoll`.
- B. `select()`.
- C. `select()/poll()`.
- D. `uWSGI`.



Отправьте в чат верную букву

Вопросы для проверки

Какой API мультиплексированного ввода-вывода планировался как более эффективная замена вызовам `select()` и `poll()` и позволяет обрабатывать большое количество одновременных соединений со сложностью $O(1)$?

A. `epoll`.

B. `select()`.

C. `select()/poll()`.

D. `uWSGI`.

Вопросы для проверки

WSGI (Web Server Gateway Interface) является стандартом взаимодействия для программ, написанных на каком языке программирования?

- A. PHP.
- B. Java.
- C. Python.
- D. C++.



Отправьте в чат верную букву

Вопросы для проверки

WSGI (Web Server Gateway Interface) является стандартом взаимодействия для программ, написанных на каком языке программирования?

- A. PHP.
- B. Java.
- C. Python.**
- D. C++.

Вопросы для проверки

Сколько обязательных параметров должен принимать вызываемый (callable) объект (функция или метод) в стандарте WSGI?

- A. Один (словарь переменных окружения).
- B. Два (словарь переменных окружения и обработчик запроса).
- C. Три (запрос, ответ и заголовки).
- D. Четыре (метод, путь, тело и заголовки).



Отправьте в чат верную букву



Вопросы для проверки

Сколько обязательных параметров должен принимать вызываемый (callable) объект (функция или метод) в стандарте WSGI?

- A. Один (словарь переменных окружения).
- B. Два (словарь переменных окружения и обработчик запроса).**
- C. Три (запрос, ответ и заголовки).
- D. Четыре (метод, путь, тело и заголовки).

Вопросы для проверки

Каков первый шаг при установлении соединения Websocket?

- A. Отправка первого фрейма данных (0x00 <UTF8 payload> 0xff).
- B. Немедленная отправка запроса на сервер для запуска процесса ожидания.
- C. HTTP handshake на подтверждение поддержки websocket.
- D. Выполнение JavaScript для открытия соединения с другого домена.



Отправьте в чат верную букву



Вопросы для проверки

Каков первый шаг при установлении соединения Websocket?

- A. Отправка первого фрейма данных (0x00 <UTF8 payload> 0xff).
- B. Немедленная отправка запроса на сервер для запуска процесса ожидания.
- C. HTTP handshake на подтверждение поддержки websocket.**
- D. Выполнение JavaScript для открытия соединения с другого домена.

Домашнее задание

1. Создать стенд с рабочими веб-приложениями на 3 разных стеках, например:
 - Nginx + PHP-FPM (wordpress/laravel) + Python (flask/django) + JS (node.js)
 - Nginx + Java (tomcat/jetty/netty) + Go + Ruby (Ruby on rails)
2. Реализации на выбор:
 - на хостовой системе через конфиги в /etc;
 - деплой через docker-compose.
3. Описать процесс запуска и тестирования в README репозитория.

В чат ДЗ отправьте ссылку на ваш git-репозиторий.



Сроки выполнения: указаны в личном кабинете



Что мы изучили?

Подведем итоги

1. Архитектуру веб-приложения
2. Варианты взаимодействия между серверами
3. Клиентские протоколы в вебе
4. Методы обработки соединений

Список материалов для изучения

1. <https://habr.com/ru/articles/259403/>
2. <http://nginx.org/ru/docs/events.html>
3. <https://habr.com/ru/articles/416669/>
4. <https://www.nginx.com/blog/websocket-nginx/>
5. <https://www.ibm.com/support/pages/flowchart-tcp-connections-and-their-definition>
6. <https://www.digitalocean.com/community/tutorials/how-to-install-wordpress-with-docker-compose-ru>
7. <https://unit.nginx.org/>



Вопросы?



Ставим "+",
если вопросы есть



Ставим "-",
если вопросов нет

Рефлексия

Цели вебинара

Проверка достижения целей

1. Понимать варианты взаимодействия компонентов веб-приложения
2. Выбирать эффективную архитектуру
3. Различать методы обработки соединений

Рефлексия



Что было самым полезным на занятии?



Как будете применять на практике то, что узнали на вебинаре?

**Заполните, пожалуйста,
опрос о занятии
по ссылке в чате**

Спасибо за внимание!

Приходите на следующие вебинары



Лавлинский Николай

Технический директор “Метод Лаб”

https://t.me/methodlab_tg
<https://www.methodlab.ru/>
<https://rutube.ru/channel/24617406/>
<https://www.youtube.com/c/NickLavlinsky>
https://www.youtube.com/@site_support
<https://vkvideo.ru/@methodlab>
<https://habr.com/ru/users/Nickmob/>

